

Inżynieria danych

Wykład 1: podstawowe pojęcia

Katarzyna Grygiel

semestr letni 2025/2026

Wykłady

- termin:
 - wtorki 14:15–15:45
- punktacja:
 - zadania 28%
 - kolokwium 12%
 - projekt 25%
 - egzamin 35%

Laboratoria

- terminy:
 - wtorki 8:30–10:00
 - wtorki 10:15–11:45
 - wtorki 12:30–14:00
- punktacja:
 - zadania i kolokwium 40/65 punktów
 - projekt 25/65 punktów

Skala ocen	
(90, 100]	bdb
(80, 90]	db+
(70, 80]	db
(60, 70]	dst+
(50, 60]	dst
[0, 50]	ndst

Bazy danych i zarządzanie nimi

Baza danych to zorganizowany zbiór powiązanych ze sobą informacji (danych).

Bazy danych i zarządzanie nimi

Baza danych to zorganizowany zbiór powiązanych ze sobą informacji (danych).

System zarządzania bazą danych (*database-management system, DBMS*) to oprogramowanie umożliwiające dostęp do danych i zarządzanie nimi. Głównym celem SZBD jest zapewnienie takiego sposobu przechowywania i przetwarzania informacji, który jest jednocześnie wygodny, wydajny i bezpieczny.

Bazy danych i zarządzanie nimi

Baza danych to zorganizowany zbiór powiązanych ze sobą informacji (danych).

System zarządzania bazą danych (*database-management system, DBMS*) to oprogramowanie umożliwiające dostęp do danych i zarządzanie nimi. Głównym celem SZBD jest zapewnienie takiego sposobu przechowywania i przetwarzania informacji, który jest jednocześnie wygodny, wydajny i bezpieczny.

Systemem bazy danych (*database system*) nazwiemy bazę danych wraz z systemem zarządzania bazą danych.

Przykłady zastosowań baz danych

- Bankowość: obsługa klienta, zarządzanie kontami, transakcje płatnościowe, naliczanie odsetek...

Przykłady zastosowań baz danych

- Bankowość: obsługa klienta, zarządzanie kontami, transakcje płatnościowe, naliczanie odsetek...
- Sklepy internetowe: informacje o produktach i klientach, tworzenie koszyków, realizacja zamówień, przyznawanie rabatów...

Przykłady zastosowań baz danych

- Bankowość: obsługa klienta, zarządzanie kontami, transakcje płatnościowe, naliczanie odsetek...
- Sklepy internetowe: informacje o produktach i klientach, tworzenie koszyków, realizacja zamówień, przyznawanie rabatów...
- USOS: informacje o studentach i pracownikach uniwersytetu, harmonogram zajęć, protokoły, ankiety...

Przykłady zastosowań baz danych

- Bankowość: obsługa klienta, zarządzanie kontami, transakcje płatnościowe, naliczanie odsetek...
- Sklepy internetowe: informacje o produktach i klientach, tworzenie koszyków, realizacja zamówień, przyznawanie rabatów...
- USOS: informacje o studentach i pracownikach uniwersytetu, harmonogram zajęć, protokoły, ankiety...
- ResearchGate: informacje o naukowcach i publikacjach, rekomendacje, oferty pracy w branży naukowej...

Przykłady zastosowań baz danych

- Bankowość: obsługa klienta, zarządzanie kontami, transakcje płatnościowe, naliczanie odsetek...
- Sklepy internetowe: informacje o produktach i klientach, tworzenie koszyków, realizacja zamówień, przyznawanie rabatów...
- USOS: informacje o studentach i pracownikach uniwersytetu, harmonogram zajęć, protokoły, ankiety...
- ResearchGate: informacje o naukowcach i publikacjach, rekomendacje, oferty pracy w branży naukowej...
- OpenStreetMap: mapy, planowanie przejazdu, zdjęcia lotnicze, tworzenie map...

Wielkie bazy danych

- Biblioteka Kongresu Stanów Zjednoczonych
 - ↪ ponad 40 milionów książek, 14 milionów zdjęć, 5,6 miliona map, 74 miliony manuskryptów, łącznie ponad 173 miliony dokumentów

Wielkie bazy danych

- Biblioteka Kongresu Stanów Zjednoczonych
 - ↪ ponad 40 milionów książek, 14 milionów zdjęć, 5,6 miliona map, 74 miliony manuskryptów, łącznie ponad 173 miliony dokumentów
- Wielki Zderzacz Hadronów
 - ↪ przetwarzanie gigantycznej ilości danych, obecnie ok. 90 petabajtów rocznie

Wielkie bazy danych

- Biblioteka Kongresu Stanów Zjednoczonych
 - ↪ ponad 40 milionów książek, 14 milionów zdjęć, 5,6 miliona map, 74 miliony manuskryptów, łącznie ponad 173 miliony dokumentów
- Wielki Zderzacz Hadronów
 - ↪ przetwarzanie gigantycznej ilości danych, obecnie ok. 90 petabajtów rocznie
- LexisNexis
 - ↪ system informacyjny służący do gromadzenia, systematyzowania, przetwarzania i wyszukiwania informacji prawnych; 250 terabajtów informacji o Amerykanach

Wielkie bazy danych

- Biblioteka Kongresu Stanów Zjednoczonych
 - ↪ ponad 40 milionów książek, 14 milionów zdjęć, 5,6 miliona map, 74 miliony manuskryptów, łącznie ponad 173 miliony dokumentów
- Wielki Zderzacz Hadronów
 - ↪ przetwarzanie gigantycznej ilości danych, obecnie ok. 90 petabajtów rocznie
- LexisNexis
 - ↪ system informacyjny służący do gromadzenia, systematyzowania, przetwarzania i wyszukiwania informacji prawnych; 250 terabajtów informacji o Amerykanach
- The World Data Centre for Climate
 - ↪ 220 terabajtów danych dostępnych w Internecie, 110 terabajtów symulacji klimatycznych, ponad 8 petabajtów danych przechowywanych na dyskach

Płaskie pliki

Płaski plik to prosta organizacja danych, będąca sekwencją zapisów nieposiadających wewnętrznej ani zewnętrznej struktury.

Aby wyszukać w nim rekord, należy przejrzeć plik od początku.

Tak zapisywane były dane w czasach, gdy nie było jeszcze komercyjnych systemów baz danych.

Płaskie pliki

Płaski plik to prosta organizacja danych, będąca sekwencją zapisów nieposiadających wewnętrznej ani zewnętrznej struktury.

Aby wyszukać w nim rekord, należy przejrzeć plik od początku.

Tak zapisywane były dane w czasach, gdy nie było jeszcze komercyjnych systemów baz danych.

Przykład: pliki **CSV** (*comma-separated values*), oddzielne dla różnych typów obiektów. Służą do zapisywania danych w plikach tekstowych i mogą być przydatne w przenoszeniu danych.

Przykład...

Autorzy(imię i nazwisko, kraj, rok_ur)

"Neil Gaiman", "Wielka Brytania", 1960

"Terry Pratchett", "Wielka Brytania", 1948

"Henryk Sienkiewicz", "Polska", 1846

Książki(tytuł, autor, rok)

"Dobry omen", "Neil Gaiman, Terry Pratchett", 1990

"Amerykańscy bogowie", "Neil Gaiman", 1960

"Quo Vadis", "Henryk Sienkiewicz", 1896

Przykład...

Autorzy(imię i nazwisko, kraj, rok_ur)

"Neil Gaiman", "Wielka Brytania", 1960

"Terry Pratchett", "Wielka Brytania", 1948

"Henryk Sienkiewicz", "Polska", 1846

Książki(tytuł, autor, rok)

"Dobry omen", "Neil Gaiman, Terry Pratchett", 1990

"Amerykańscy bogowie", "Neil Gaiman", 1960

"Quo Vadis", "Henryk Sienkiewicz", 1896

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Przykład...

```
Autorzy(imię i nazwisko, kraj, rok_ur)
```

```
"Neil Gaiman", "Wielka Brytania", 1960
```

```
"Terry Pratchett", "Wielka Brytania", 1948
```

```
"Henryk Sienkiewicz", "Polska", 1846
```

```
Książki(tytuł, autor, rok)
```

```
"Dobry omen", "Neil Gaiman, Terry Pratchett", 1990
```

```
"Amerykańscy bogowie", "Neil Gaiman", 1960
```

```
"Quo Vadis", "Henryk Sienkiewicz", 1896
```

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

```
for line in file:
```

```
    record = parse(line)
```

```
    if record[2] > 1900:
```

```
        print(record[0])
```

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

Jak uwzględnić fakt, że Sienkiewicz pisał pod różnymi pseudonimami?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

Jak uwzględnić fakt, że Sienkiewicz pisał pod różnymi pseudonimami?

Co się stanie, jeśli któraś dana będzie wpisana w innym formacie?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

Jak uwzględnić fakt, że Sienkiewicz pisał pod różnymi pseudonimami?

Co się stanie, jeśli któraś dana będzie wpisana w innym formacie?

Czy trzeba kopiować wszystkie dane, jeśli dwie aplikacje będą korzystać z tych samych danych i zapytań?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

Jak uwzględnić fakt, że Sienkiewicz pisał pod różnymi pseudonimami?

Co się stanie, jeśli któraś dana będzie wpisana w innym formacie?

Czy trzeba kopiować wszystkie dane, jeśli dwie aplikacje będą korzystać z tych samych danych i zapytań?

Co się stanie, gdy dwie aplikacje będą próbowały równocześnie zmienić plik?

...i problemy

Jak znaleźć tytuły wszystkich książek napisanych po XIX wieku?

Jak znaleźć książki, których autorem jest tylko Pratchett?

Jak uwzględnić fakt, że Sienkiewicz pisał pod różnymi pseudonimami?

Co się stanie, jeśli któraś dana będzie wpisana w innym formacie?

Czy trzeba kopiować wszystkie dane, jeśli dwie aplikacje będą korzystać z tych samych danych i zapytań?

Co się stanie, gdy dwie aplikacje będą próbowały równocześnie zmienić plik?

Co powinno się wydarzyć, jeśli nastąpiła awaria podczas modyfikacji pliku?

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,
- obsługę wielu użytkowników,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,
- obsługę wielu użytkowników,
- wygodę,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,
- obsługę wielu użytkowników,
- wygodę,
- wydajność,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,
- obsługę wielu użytkowników,
- wygodę,
- wydajność,
- niezawodność,

Czego oczekujemy od SZBD?

System zarządzania bazą danych powinien zapewniać:

- przechowywanie i przetwarzanie gigantycznych ilości danych,
- trwałość danych,
- bezpieczeństwo,
- obsługę wielu użytkowników,
- wygodę,
- wydajność,
- niezawodność,
- spójność danych.

Trochę historii

Lata 60. i 70.

Utrzymywanie bazy jest kłopotliwe ze względu na zależność danych od aplikacji: w niej właśnie zaimplementowana jest struktura bazy. Jakiegokolwiek zmiany w strukturze można wykonać usuwając dane, a następnie wczytując je ponownie.

W roku 1970 Ted Codd wprowadza model relacyjny, publikując artykuł *A Relational Model of Data for Large Shared Data Banks*.

Proponuje trzynaście postulatów, wśród których są następujące.

1. Dane powinny być przechowywane w prostych strukturach.
2. Dostęp do nich powinien być możliwy za pomocą języka wysokiego poziomu.
3. Dane powinny być niezależne zarówno w warstwie fizycznej, jak i logicznej.

Za swoją pracę Codd otrzymuje w 1981 r. **Nagrodę Turinga**.

Trochę historii

Przełom lat 70. i 80.

Z początku bazy relacyjne nie sprawdzają się w praktyce: ich wydajność jest gorsza niż istniejących baz hierarchicznych i sieciowych.

Zmienia się to na początku lat 70., gdy powstają pierwsze efektywne relacyjne SZBD. W 1973 roku w Berkeley powstaje Ingres, dwa lata później IBM projektuje system R, a w latach 1977–1979 startuje Oracle.

W 1976 Peter Chen publikuje artykuł *The Entity-Relationship Model – Toward a Unified View of Data*, wprowadzając model związków encji.

Trochę historii

Lata 80. i 90.

Systemy relacyjne stają się dominujące, pozwalając programistom pracować na poziomie logicznym bez odwoływania się do szczegółów implementacyjnych. Powstają pierwsze prace dotyczące rozproszonych baz danych i baz obiektowych.

W latach 90. powstaje SQL oraz narzędzia pozwalające na analizę ogromnych ilości danych.

Rozwój Internetu wymusza na systemach bazodanowych wsparcie dla olbrzymiej liczby transakcji, niezawodność i konieczność dostępu dla wszystkich użytkowników przez całą dobę. Poza tym systemy muszą wspierać interfejsy webowe dla danych.

Trochę historii

2000–

Pojawia się wiele nowych typów danych, w szczególności na znaczeniu zyskują dane semistrukturalne. Standardami przy przesyłaniu danych do aplikacji stają się XML, JSON czy YAML. Wsparcie tych formatów pojawia się w systemach relacyjnych.

Wsparcia wymagają również dane przestrzenne, zawierające informacje geograficzne wykorzystywane w systemach nawigacyjnych. Rozwój sieci społecznościowych przyczynia się do powstania baz grafowych.

Współczesne możliwości obliczeniowe sprawiają, że upowszechniają się systemy bazodanowe projektowane z myślą o eksploracji danych i przetwarzaniu analitycznym.

Podział systemów baz danych

Podziału systemów bazodanowych możemy dokonać ze względu na różne kryteria:

- model danych,

Co właściwie chcemy reprezentować? Jaka jest struktura naszych danych? Jakie są między nimi zależności? Jakie ograniczenia musimy zamodelować?

Podział systemów baz danych

Podziału systemów bazodanowych możemy dokonać ze względu na różne kryteria:

- model danych,

Co właściwie chcemy reprezentować? Jaka jest struktura naszych danych? Jakie są między nimi zależności? Jakie ograniczenia musimy zamodelować?

- cel wykorzystania,

Jakie operacje będą wykonywane? Czy będziemy dokonywać częstych zmian w bazie czy może dominować będą odczyty?

Podział systemów baz danych

Podziału systemów bazodanowych możemy dokonać ze względu na różne kryteria:

- **model danych,**

Co właściwie chcemy reprezentować? Jaka jest struktura naszych danych? Jakie są między nimi zależności? Jakie ograniczenia musimy zamodelować?

- **cel wykorzystania,**

Jakie operacje będą wykonywane? Czy będziemy dokonywać częstych zmian w bazie czy może dominować będą odczyty?

- **wolumen danych i liczba użytkowników,**

Czy nasza baza będzie obsługiwana przez jednego użytkownika czy przez wielu? Czy wszystkie dane dają się zapisać w jednym, stosunkowo niewielkim pliku, czy może chodzi o terabajty informacji?

Podział systemów baz danych

Podziału systemów bazodanowych możemy dokonać ze względu na różne kryteria:

- model danych,

Co właściwie chcemy reprezentować? Jaka jest struktura naszych danych? Jakie są między nimi zależności? Jakie ograniczenia musimy zamodelować?

- cel wykorzystania,

Jakie operacje będą wykonywane? Czy będziemy dokonywać częstych zmian w bazie czy może dominować będą odczyty?

- wolumen danych i liczba użytkowników,

Czy nasza baza będzie obsługiwana przez jednego użytkownika czy przez wielu? Czy wszystkie dane dają się zapisać w jednym, stosunkowo niewielkim pliku, czy może chodzi o terabajty informacji?

- sposób przechowywania danych.

Czy cała baza będzie znajdować się na jednym urządzeniu czy będzie istnieć fizycznie na wielu maszynach?

Ile danych? Ile maszyn?

Podział ze względu na ilość danych:

- wbudowane systemy zarządzania bazami danych,

Obsługują bardzo małe bazy, są wbudowane w aplikację, sprawdzają się w przypadku lokalnych aplikacji wykorzystywanych przez jednego użytkownika, działają szybko i wydajnie, nie nadają się w sytuacji, gdy dokonywanych jest wiele zapisów.

- hurtownie danych.

Są to rozbudowane bazy danych przechowujące olbrzymie ilości danych, przeprowadzane operacje mają charakter analityczny, a nie transakcyjny.

Ile danych? Ile maszyn?

Podział ze względu na ilość danych:

- wbudowane systemy zarządzania bazami danych,

Obsługują bardzo małe bazy, są wbudowane w aplikację, sprawdzają się w przypadku lokalnych aplikacji wykorzystywanych przez jednego użytkownika, działają szybko i wydajnie, nie nadają się w sytuacji, gdy dokonywanych jest wiele zapisów.

- hurtownie danych.

Są to rozbudowane bazy danych przechowujące olbrzymie ilości danych, przeprowadzane operacje mają charakter analityczny, a nie transakcyjny.

Podział ze względu na liczbę węzłów:

- bazy scentralizowane,

Cechuje je wysoka przepustowość, większa dostępność, a także niewielkie opóźnienia, problemem jest zapewnienie bezpieczeństwa i odporności na manipulacje.

- bazy rozproszone.

Zapewniają większą prywatność oraz bezpieczeństwo kosztem możliwego przeciążenia sieci i opóźnień transakcji.

Dwa główne cele stosowania baz danych

- **Przetwarzanie transakcyjne** (*online transaction processing*):
system wspomaga korzystanie z bazy danych przez wielu użytkowników, z których każdy odbiera względnie małą ilość danych i dokonuje odczytu lub niewielkich modyfikacji.

Dwa główne cele stosowania baz danych

- **Przetwarzanie transakcyjne** (*online transaction processing*):
system wspomaga korzystanie z bazy danych przez wielu użytkowników, z których każdy odbiera względnie małą ilość danych i dokonuje odczytu lub niewielkich modyfikacji.
- **Przetwarzanie analityczne** (*online analytical processing*):
system współpracuje z narzędziami eksploracji danych (*data mining*) i systemami wspomagania decyzji, rozbudowanych obecnie do zaawansowanych systemów analityki biznesowej (*business intelligence*). Liczba użytkowników jest stosunkowo niewielka, operacje najczęściej ograniczają się do odczytu danych.

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,
- związków encji,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,
- związków encji,
- obiektowy,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,
- związków encji,
- obiektowy,
- NoSQL:
 - klucz–wartość,
 - grafowy,
 - rodzina kolumn,
 - dokument,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,
- związków encji,
- obiektowy,
- NoSQL:
 - klucz–wartość,
 - grafowy,
 - rodzina kolumn,
 - dokument,
- przestrzenny,
- temporalny,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

- relacyjny,
- związków encji,
- obiektowy,
- NoSQL:
 - klucz–wartość,
 - grafowy,
 - rodzina kolumn,
 - dokument,
- przestrzenny,
- temporalny,
- hierarchiczny,
- sieciowy,

Model danych

Istnieje wiele modeli danych, w tym między innymi:

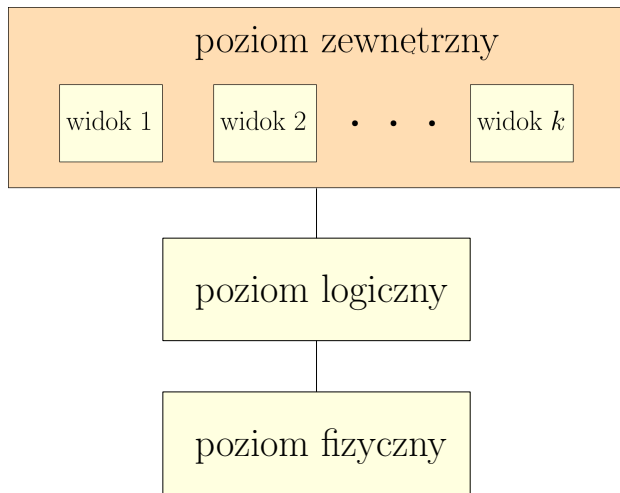
- relacyjny,
- związków encji,
- obiektowy,
- NoSQL:
 - klucz–wartość,
 - grafowy,
 - rodzina kolumn,
 - dokument,
- przestrzenny,
- temporalny,
- hierarchiczny,
- sieciowy,
- ...

Poziomy abstrakcji danych

Aby ułatwić użytkownikom korzystanie z systemu bazodanowego, dostęp do danych jest wielopoziomowy.

- Najniższy poziom to **poziom fizyczny** (*physical level*), który określa, w jaki sposób dane są przechowywane na dysku oraz szczegóły implementacyjne struktur danych.
- Pośredni poziom to **poziom logiczny** (*logical level*), który określa, jakie dane znajdują się w bazie oraz jakie zachodzą między nimi związki. Już na tym etapie dla użytkownika nie są istotne szczegóły implementacyjne wykorzystywanych struktur.
- Ostatni poziom to **poziom zewnętrzny** (*view level*), który określa sposób użycia bazy danych przez użytkowników końcowych. System może zapewniać różne widoki dla różnych użytkowników.

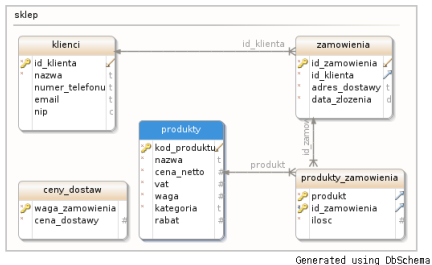
Poziomy abstrakcji danych



Schematy

Stan bazy to zbiór informacji w bazie w konkretnym momencie, **schemat** opisuje natomiast jej ogólną konstrukcję.

Schemat fizyczny bazy danych opisuje projekt bazy danych na poziomie fizycznym, natomiast **schemat logiczny** – na poziomie logicznym. Baza danych na poziomie zewnętrznym może mieć wiele schematów, zwanych **podszchematami**, które opisują jej różne widoki.



Przykład schematu logicznego

Języki baz danych i SQL

Interakcję z bazą danych zapewniają języki:

- **definicji danych** (*data-definition language, DDL*),
- **manipulacji danymi** (*data-manipulation language, DML*),
- **zapytań** (*data-query language, DQL*),
- **kontroli nad danymi** (*data-control language, DCL*).

Języki baz danych i SQL

Interakcję z bazą danych zapewniają języki:

- **definicji danych** (*data-definition language, DDL*),
- **manipulacji danymi** (*data-manipulation language, DML*),
- **zapytań** (*data-query language, DQL*),
- **kontroli nad danymi** (*data-control language, DCL*).

Zazwyczaj nie są to różne języki, a stanowią część jednego języka. W przypadku relacyjnych baz danych jest to praktycznie zawsze **SQL** (*Structured Query Language*).

SQL jest deklaratywnym językiem wysokiego poziomu (tzn. specyfikuje strukturę wyniku, a nie sposób jego realizacji).

Ostatnie standardy: SQL-2008, SQL-2011, SQL-2016, SQL-2023.

DDL

Język definicji danych służy do zdefiniowania schematu logicznego oraz określenia własności danych, takich jak dziedzina danych (typ, zakres długości, format) czy integralność referencyjna (utrzymanie powiązań między tabelami).

```
CREATE TABLE klienci (  
    id          NUMERIC(6) PRIMARY KEY,  
    nazwa       VARCHAR(250) NOT NULL,  
    telefon     VARCHAR(25),  
    email       VARCHAR(100)  
);
```

Najważniejsze polecenia: CREATE, ALTER, DROP.

Język manipulacji danymi służy do ich umieszczania w bazie, usuwania, przeglądania oraz dokonywania zmian.

```
INSERT INTO klienci VALUES  
    (1, 'Jan K.', NULL, 'jan@klient.pl'),  
    (2, 'Anna N.', '123456789', NULL);
```

```
UPDATE klienci  
SET     telefon = '987654321'  
WHERE   nazwa = 'Jan K.';
```

Najważniejsze polecenia: INSERT, UPDATE, DELETE.

W zakres języka zapytań do bazy wchodzi tylko jedno polecenie — SELECT, które zwraca zestawienie danych.

```
SELECT * FROM klienci;
```

```
SELECT nazwa, telefon  
FROM klienci  
WHERE telefon IS NOT NULL  
ORDER BY nazwa;
```

```
SELECT nazwa, telefon  
INTO telefony  
FROM klienci  
WHERE telefon IS NOT NULL;
```

Język kontroli nad danymi służy do nadawania uprawnień użytkownikom do obiektów bazodanowych, przypisywania ról, zmieniania haseł.

```
REVOKE INSERT ON oceny FROM student01;
```

```
GRANT ALL PRIVILEGES  
    ON tajne_dane  
    TO agent007  
    WITH GRANT OPTION;
```

Najważniejsze polecenia: GRANT, REVOKE.

Projektowanie bazy: faza conceptualna

Projektowanie bazy danych dotyczy głównie projektowania schematu. Początkowa faza zakłada zebranie informacji i specyfikację potrzeb i wymagań przyszłych użytkowników bazy.

Na podstawie analizy dotyczącej wymagań i funkcjonalności, niezależnie od szczegółów implementacyjnych, tworzy się model conceptualny.

Projektowanie bazy: faza koncepcyjna

Projektowanie bazy danych dotyczy głównie projektowania schematu. Początkowa faza zakłada zebranie informacji i specyfikację potrzeb i wymagań przyszłych użytkowników bazy.

Na podstawie analizy dotyczącej wymagań i funkcjonalności, niezależnie od szczegółów implementacyjnych, tworzy się model koncepcyjny.

W przypadku modelu relacyjnego należy:

- ustalić występujące zbiory encji,
- ustalić występujące związki i ich typy,
- określić atrybuty poszczególnych encji i związków,
- podać dziedziny tych atrybutów,
- ustalić klucze relacji,
- zweryfikować otrzymany model pod kątem występowania redundancji oraz możliwości realizowania transakcji.

Projektowanie bazy: fazy logiczna i fizyczna

Kolejna faza polega na transformacji modelu konceptualnego w model logiczny.

W przypadku modelu relacyjnego na tym etapie należy:

- wyznaczyć relacje odpowiadające encjom i związkom z modelu konceptualnego,
- znormalizować relacje,
- wyznaczyć więzy integralnościowe,
- w przypadku tworzenia oddzielnych modeli dla większej liczby użytkowników utworzyć model globalny.

Projektowanie bazy: fazy logiczna i fizyczna

Kolejna faza polega na transformacji modelu konceptualnego w model logiczny.

W przypadku modelu relacyjnego na tym etapie należy:

- wyznaczyć relacje odpowiadające encjom i związkom z modelu konceptualnego,
- znormalizować relacje,
- wyznaczyć więzy integralnościowe,
- w przypadku tworzenia oddzielnych modeli dla większej liczby użytkowników utworzyć model globalny.

Ostatni etap to implementacja bazy danych z uwzględnieniem relacji, organizacji plików, indeksów (czyli struktur danych stosowanych w celu efektywnego dostępu do danych), więzów integralnościowych i zagadnień związanych z bezpieczeństwem.

Główne składowe SZBD

Za wykonywanie poszczególnych zadań w obrębie SZDB odpowiedzialnych jest kilka modułów:

- **moduł zarządzania pamięcią**, wybierający właściwe dane z pamięci,
- **moduł zarządzania zapytaniami**, przekształcający zapytania w ciąg poleceń żądających dostarczenia określonych danych,
- **moduł zarządzania transakcjami**, odpowiadający za poprawność przetwarzania transakcji.

Moduł zarządzania pamięcią

Moduł zarządzania pamięcią dzieli się na następujące składowe:

- **moduł zarządzania autoryzacją i integralnością danych**, sprawdzający, czy spełnione są wszystkie ograniczenia nałożone na dane oraz kontrolujący autoryzację użytkowników;
- **moduł zarządzania plikami**, przechowujący dane o miejscu zapisania plików na dysku i struktury danych wykorzystywane do reprezentowania informacji przechowywanych na dysku;
- **moduł zarządzania buforami**, odpowiedzialny za przekazywanie danych z dysku do pamięci operacyjnej i decyzje, które przechowywać w pamięci podręcznej.

Ponadto moduł ten tworzy pliki danych (przechowujące dane), słownik danych (przechowujący metadane o strukturze bazy) oraz indeksy.

Moduł zarządzania zapytaniami

Moduł zarządzania zapytaniami dzieli się na następujące składowe:

- **interpreter języka DDL**, interpretujący polecenia w języku definicji danych i zapisujący odpowiednie metadane w słowniku;
- **kompilator języka DML**, tworzący plany zapytań dla poleceń w języku manipulacji danymi, wybierający optymalny plan i tłumaczący go na język niskiego poziomu;
- **moduł wykonujący zapytania**, realizujący niskopoziomowe instrukcje przekazywane przez kompilator DML.

Moduł zarządzania transakcjami i ACID

Moduł zarządzania transakcjami zapewnia poprawność przeprowadzenia transakcji. Poprawność oznacza w tym wypadku:

- **niepodzielność** (*atomicity*), czyli albo cała transakcja została wykonana, albo żaden z jej elementów nie został uwzględniony;
- **spójność** (*consistency*), czyli po zakończeniu transakcji żaden z warunków narzuconych na bazę nie jest naruszony;
- **izolację** (*isolation*), czyli żadne dwie transakcje przetwarzane równocześnie nie wpływają wzajemnie na siebie;
- **trwałość** (*durability*), czyli po zakończeniu transakcji jej wynik nie zostanie utracony z powodu awarii systemu.

Grupy użytkowników

- Administratorzy baz danych
- Administratorzy danych
- Projektanci baz danych
- Twórcy aplikacji
- Użytkownicy końcowi

Administratorzy baz danych

Administratorzy baz danych:

- odpowiadają za fizyczną realizację bazy danych, kontrolę bezpieczeństwa i spójności, zapewnienie sprawnego działania aplikacji użytkowników, wydajność i konserwację systemu,
- opiekują się gotowymi aplikacjami baz danych, instalują je, zapewniają ciągłość pracy aplikacji,
- uruchamiają, zatrzymują, restartują bazę danych,
- odpowiadają za ciągłość pracy oraz wydajność bazy danych,
- tworzą kopie zapasowe, przywracają i naprawiają bazę danych,
- zarządzają użytkownikami i nadają im uprawnienia,
- odpowiadają za zapewnienie optymalnych warunków dostępu do bazy danych na zasadach bezpieczeństwa określonych odpowiednią polityką dostępu do bazy danych.

Zaawansowani użytkownicy

Administratorzy danych:

- podejmują decyzje, jakie dane powinny być przechowywane w bazie danych,
- projektują i kontrolują ustalone standardy,
- odpowiadają za prawidłowy rozwój bazy.

Projektanci logicznej bazy danych:

- analizują dane,
- tworzą związki między danymi i zarządzają nimi,
- określają ograniczenia dla danych przechowywanych w bazie.

Projektanci fizycznej bazy danych:

- odwzorowują logiczny projekt bazy danych w postaci tabel i więzów integralności,
- wybierają odpowiednie struktury i metody dostępu do danych,
- projektują procedury gwarantujące bezpieczeństwo danych.

Programiści i użytkownicy końcowi

Twórcy aplikacji:

- tworzą programy, które pozwalają użytkownikom w prosty sposób realizować operacje w bazie.

Użytkownicy profesjonalni:

- znają strukturę bazy i funkcje realizowane przez SZBD,
- mogą używać języka wysokiego poziomu (np. SQL) w celu wykonania potrzebnych operacji.

Użytkownicy naiwni (laicy):

- na ogół nie są świadomi istnienia SZBD,
- komunikują się z bazą korzystając z programów pozwalających w prosty sposób realizować konieczne operacje.

Podsumowanie

Zalety SZBD:

- kontrola redundancji,
- zabezpieczenia przed nieautoryzowanymi użytkownikami,
- współbieżność dostępu,
- trwałość danych, możliwość ich odzyskiwania i przechowywania kopii zapasowych,
- struktury i techniki pozwalające na efektywny dostęp do danych i ich przetwarzanie,
- reprezentacja złożonych zależności między danymi,
- ograniczenia integralnościowe,
- niezależność danych od działających na nich programów.

Wady SZBD:

- początkowa inwestycja: zasoby ludzkie, architektura, czas,
- duża złożoność,
- koszty związane z zarządzaniem i konserwacją systemu.

Najpopularniejsze SZBD

429 systems in ranking, March 2026

Rank			DBMS	Database Model	Score		
Mar 2026	Feb 2026	Mar 2025			Mar 2026	Feb 2026	Mar 2025
1.	1.	1.	Oracle	Relational, Multi-model 	1182.46	-21.05	-70.62
2.	2.	2.	MySQL	Relational, Multi-model 	858.34	-9.88	-129.79
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model 	711.47	+3.33	-76.67
4.	4.	4.	PostgreSQL	Relational, Multi-model 	680.08	+8.05	+16.66
5.	5.	5.	MongoDB	Document, Multi-model 	383.58	+4.85	-12.85
6.	6.	6.	Snowflake	Relational	211.24	+3.10	+49.46
7.	 8.	 13.	Databricks	Multi-model 	145.81	+1.29	+49.80
8.	 7.	 7.	Redis	Key-value, Multi-model 	145.19	-1.85	-10.17
9.	9.	9.	IBM Db2	Relational, Multi-model 	111.38	+0.16	-15.19
10.	10.	 8.	Elasticsearch	Multi-model 	103.58	-2.88	-27.80
11.	11.	11.	Apache Cassandra	Wide column, Multi-model 	101.88	+0.28	-4.78
12.	12.	 10.	SQLite	Relational	95.97	-3.22	-17.11
13.	13.	 14.	MariaDB 	Relational, Multi-model 	87.00	+0.91	-7.23
14.	14.	 17.	Microsoft Azure SQL Database	Relational, Multi-model 	73.92	+0.26	+0.01
15.	 16.	 18.	Apache Hive	Relational	72.95	-0.05	+2.87
16.	 15.	 15.	Splunk	Search engine	72.21	-0.84	-6.66
17.	17.	 12.	Microsoft Access	Relational	67.56	-1.90	-29.15
18.	18.	 16.	Amazon DynamoDB	Multi-model 	65.42	-2.46	-11.16
19.	19.	19.	Google BigQuery	Relational	55.82	-3.76	-2.14
20.	20.	20.	Neo4j	Graph	47.32	-1.94	+1.17



February 26, 2026: PostgreSQL 18.3, 17.9, 16.13, 15.17, and 14.22 Released!

PostgreSQL: The World's Most Advanced Open Source Relational Database

[Download →](#)[New to PostgreSQL?](#)

New to PostgreSQL?

PostgreSQL is a powerful, open source object-relational database system with over 35 years of active development that has earned it a strong reputation for reliability, feature robustness, and performance.

There is a wealth of information to be found describing how to [install](#) and [use](#) PostgreSQL through the [official documentation](#). The [open source community](#) provides many helpful places to become familiar with PostgreSQL, discover how it works, and find career opportunities. Learn more on how to [engage with the community](#).

[Learn More](#)[Feature Matrix](#)[Governance](#)

Latest Releases

2026-02-26 - PostgreSQL 18.3, 17.9, 16.13, 15.17, and 14.22 Released!

The PostgreSQL Global Development Group has [released an update](#) to all supported versions of PostgreSQL, including 18.3, 17.9, 16.13, 15.17, and 14.22. This is an out-of-cycle release that fixes [several regressions](#) reported after the last update release.

For the more information about this release, please review the [release notes](#). You can download PostgreSQL from the [download](#) page.

For more Information on PostgreSQL 18, the latest major version of PostgreSQL, please see the full press release and [translations of the release announcement](#) in the press kit.

18.3 · 2026-02-26 · [Notes](#)

17.9 · 2026-02-26 · [Notes](#)

16.13 · 2026-02-26 · [Notes](#)